

KAVA vs CODI

A Controlled and Mechanistic Study of Latent Reasoning Supervision

Proposed SProj Theme

Endpoint versus trajectory supervision for latent chain-of-thought reasoning

Central research question

Does matching a compressed teacher KV trajectory, as in KAVA, produce more accurate, robust, and causally meaningful latent reasoning than matching an answer-level hidden representation, as in CODI, when architecture, data, latent budget, and training compute are controlled?

Prepared as a research-focused semester project proposal

1. Executive Summary

A simple comparison asking whether KAVA is better than CODI would be mostly a benchmark exercise. The stronger research problem is to identify **when** one method is better, **why** it is better, and **which internal mechanism** explains the difference. CODI and KAVA represent two distinct supervision paradigms for latent reasoning. CODI mainly aligns the teacher and student at an answer-related hidden state, whereas KAVA aligns the student's latent trajectory with a compressed version of the teacher's KV-cache across reasoning steps and layers.

The proposed project therefore studies the trade-off between **flexible endpoint supervision** and **dense trajectory supervision**. It will compare the methods under a unified training and evaluation pipeline, then investigate their internal representations using probing, representational similarity, latent-step ablation, and optionally activation patching.

Dimension	CODI	KAVA
Supervision target	Answer-level hidden representation	Compressed KV trajectory
Supervision density	Relatively sparse / endpoint-oriented	Dense / stepwise across layers
Potential advantage	Greater flexibility and simpler optimization	Richer guidance for long or complex reasoning
Potential weakness	May under-supervise intermediate latent steps	May over-constrain the student or inherit teacher artifacts

2. Conceptual Background

2.1 What is latent reasoning?

In explicit chain-of-thought reasoning, a language model writes intermediate steps as ordinary text tokens before producing the final answer. In latent reasoning, the model instead performs part of this computation using continuous internal vectors. These vectors are not required to correspond directly to human-readable words. The attraction is that a short sequence of latent thoughts may replace a much longer textual reasoning trace.

Let the question be **Q**, the explicit reasoning trace be **C**, the latent reasoning states be **Z = {z1, ..., zM}**, and the answer be **A**. Explicit reasoning models generate $Q \rightarrow C \rightarrow A$. Latent reasoning models aim to generate $Q \rightarrow Z \rightarrow A$, where Z is much shorter than C.

2.2 Why supervision is difficult

Textual reasoning has a clear training target: each next token can be supervised with cross-entropy. Latent thoughts do not have an obvious ground-truth target. There is no unique correct vector for 'multiply 4 by 5' or 'compare the two quantities.' Therefore, latent reasoning methods need an indirect signal that teaches internal states to carry useful reasoning information.

2.3 Two supervision philosophies

Endpoint supervision: teach the student to arrive at an internal state similar to the teacher immediately before answer generation. This is the central idea behind CODI.

Trajectory supervision: teach the student's sequence of internal states to resemble a compressed version of the teacher's full reasoning memory across multiple steps and layers. This is the central idea behind KAVA.

Teacher reasoning	Latent student	Training signal
Full written CoT	$z1, z2, \dots, zM$	CODI: match answer-ready hidden state
Full written CoT and its KV-cache	$z1, z2, \dots, zM$	KAVA: match compressed keys and values step-by-step

3. Research Framing

3.1 Recommended title

Endpoint versus Trajectory Supervision for Latent Chain-of-Thought Reasoning

Alternative title: **From Endpoints to Trajectories: How Much Internal Supervision Does Latent Reasoning Need?**

3.2 Primary research question

Under matched backbones, datasets, latent-token budgets, optimization budgets, and evaluation protocols, does stepwise compressed KV-cache supervision outperform answer-level hidden-state alignment?

3.3 Supporting research questions

- **Reasoning depth:** Does KAVA gain more than CODI as the number of reasoning steps increases?
- **Trace style:** Does trajectory supervision help more for verbose natural-language CoTs than for equation-only traces?
- **Data efficiency:** Which method learns better with 10%, 25%, 50%, or 100% of the training data?
- **Out-of-distribution generalization:** Does CODI's looser supervision improve transfer to unfamiliar problems?
- **Teacher noise:** Which method is more robust to incorrect, verbose, or stylistically inconsistent teacher traces?
- **Mechanism:** Do the methods encode intermediate results differently, even when final accuracy is similar?

3.4 Why this is a research problem, not just benchmarking

The purpose is not merely to report a leaderboard. The contribution is to isolate the effect of supervision granularity and determine whether performance differences come from richer intermediate guidance, stronger alignment with explicit CoT, easier optimization, or accidental confounds such as different datasets and hyperparameters.

4. Falsifiable Hypotheses

Hypothesis	Prediction	How it could be falsified
H1: Trajectory advantage	KAVA performs better on long, multi-step, natural-language reasoning.	CODI matches or exceeds KAVA in these settings under controlled training.
H2: Endpoint sufficiency	CODI remains competitive on short and highly structured problems.	KAVA dominates even on one- or two-step equation-only tasks.
H3: Flexibility-generalization trade-off	CODI generalizes better under strong distribution shift.	KAVA retains equal or better OOD performance despite denser teacher alignment.
H4: Optimization trade-off	KAVA learns quickly but is more sensitive to loss weighting and compression quality.	KAVA is no more sensitive than CODI across seeds and hyperparameters.
H5: Teacher-noise sensitivity	KAVA inherits more trace-level noise because it distills the trajectory.	KAVA remains more robust than CODI when teacher traces are corrupted.

The project should begin without assuming that KAVA is universally better. A strong result may be a conditional conclusion such as: 'KAVA is preferable for long natural-language reasoning, while CODI is more robust or efficient for shorter and shifted tasks.'

5. Experimental Design

5.1 Backbones

- **Primary reproduction backbone:** GPT-2, because CODI reports a clear setup at this scale and training is relatively affordable.
- **Optional modern backbone:** a 0.5B-1.5B open model, used only after the unified pipeline works.
- **Rule:** use the same initialization, tokenizer, and parameter-efficient training setup for both methods.

5.2 Datasets

Role	Dataset	Purpose
Training	GSM8K-Aug	Structured equation-only CoT
Training	GSM8K-Aug-NL	Verbose natural-language CoT
In-domain test	GSM8K	Main accuracy comparison
OOD test	SVAMP	Robustness to altered word-problem structure
OOD test	MultiArith	Multi-step arithmetic transfer
OOD test	GSM-Hard	Numerical distribution shift

5.3 Fairness controls

- Same backbone and tokenizer.
- Same number of latent thoughts.
- Same data split, data order, and preprocessing.
- Same optimizer family, batch size, training steps, and precision.
- Same LoRA or full-fine-tuning policy.
- Same answer prompt and answer extraction.
- Same decoding settings.
- At least three random seeds.
- Comparable hyperparameter-tuning budget for both methods.

5.4 Core baselines

- **No-CoT-SFT:** direct question-to-answer learning.
- **CoT-SFT:** explicit textual reasoning.
- **Latent student without distillation:** tests whether extra latent tokens alone help.
- **CODI:** endpoint hidden-state alignment.
- **KAVA:** compressed KV-trajectory alignment.
- **Naive or random KV targets:** checks whether meaningful cache compression is necessary.
- **Uniformly sampled KV targets:** distinguishes compression quality from mere target density.

6. Evaluation Metrics

Category	Metrics
Task performance	Exact-match accuracy, accuracy by step count, accuracy by trace style, in-domain and OOD accuracy
Efficiency	Inference latency, peak memory, training time, latent-token count, compression ratio versus explicit CoT
Optimization	Loss curves, convergence speed, gradient norms, seed variance, sensitivity to loss weights
Robustness	Paraphrases, changed numbers, irrelevant sentences, noisy teacher traces, verbose traces
Calibration	Confidence on correct and incorrect answers, expected calibration error, uncertainty versus benefit of more latent steps

A method should not be declared 'better' from accuracy alone. A fair conclusion must distinguish accuracy, inference efficiency, training cost, stability, robustness, and interpretability.

7. Mechanistic Analysis

7.1 Linear probes

Train lightweight probes on each latent state to predict intermediate arithmetic results, operation type, final answer, or remaining reasoning depth. This tests whether reasoning information is present even when the model's own answer is wrong.

7.2 Representational similarity

Use CKA, SVCCA, or linear alignment to compare explicit-CoT teacher states with CODI and KAVA latent states. One possible outcome is that KAVA follows the teacher trajectory more closely, while CODI reaches a similar final state through a different internal route.

7.3 Latent-step ablation

Zero, replace, or shuffle individual latent states. If CODI relies mainly on the final latent state while KAVA distributes information across steps, their causal structures will differ even at matched accuracy.

7.4 Activation patching

Replace a latent state from an incorrect run with the corresponding state from a correct run. A causal change in the answer identifies which latent positions carry decisive information.

7.5 Sparse feature analysis (optional)

Train a sparse autoencoder on teacher, CODI, and KAVA activations to test whether they share features for operands, intermediate results, verification, contradiction, or answer confidence.

8. Stronger Original Extension: A Supervision-Granularity Continuum

Instead of treating CODI and KAVA as two isolated systems, create a controlled continuum of internal supervision:

- Align only the final hidden state.
- Align the final hidden state across all layers.
- Align selected intermediate latent states.
- Align only values.
- Align only keys.
- Align keys and values at selected layers.
- Align the full compressed KV trajectory.

This changes the central question from 'Which paper is better?' to:

How much, where, and what kind of internal supervision is necessary for effective latent reasoning?

A strong empirical result would identify a point of diminishing returns. For example, full trajectory alignment may not be necessary; selected-layer or value-only alignment might preserve most of KAVA's benefit at lower training cost.

9. Major Confounds and Validity Threats

Different implementations

Original repositories may use different prompts, preprocessing, schedules, answer extraction, and latent budgets. Running both codebases unchanged is not a controlled comparison.

KAVA is newer

A performance advantage may reflect later engineering choices rather than a fundamentally better supervision principle.

Unequal loss density

KAVA receives many more layer- and step-level targets. Its advantage may result from more supervision rather than better supervision.

Compression quality

KAVA depends on the teacher-cache compression method. Different eviction rules may change the quality of supervision.

Unequal tuning effort

One method may appear better because it received more hyperparameter search.

Optimization scale

Loss magnitudes and gradient behavior may differ substantially. Separate tuning is allowed, but total tuning budget must be controlled.

Final accuracy hides mechanism

Two methods can achieve the same answer accuracy while using very different internal computations.

10. Feasible Semester Plan

Phase	Weeks	Main work	Deliverable
1. Reproduction	1-4	Unify data, training, and evaluation; reproduce one CODI and one KAVA setting.	Verified training pipeline and baseline table.
2. Controlled comparison	5-8	Matched backbone, data, latent budget, compute, and three seeds.	Accuracy, efficiency, stability, and OOD results.
3. Mechanistic study	9-12	Choose one main analysis: probes, similarity, or latent-step ablation.	Mechanistic evidence explaining the observed difference.
4. Extension and writing	13-16	Run supervision-granularity ablation and consolidate findings.	SProj report, poster, code, and reproducible evaluation scripts.

Recommended minimum scope

- One backbone.
- Two training trace styles: equation-only and natural language.
- Four evaluation datasets.
- Three random seeds.
- One main mechanistic analysis.
- One supervision-granularity ablation.

11. Final Proposal

Proposed title

Endpoint versus Trajectory Supervision for Latent Chain-of-Thought Reasoning

Central question

Does matching a compressed teacher KV trajectory, as in KAVA, produce more accurate and robust latent reasoning than matching an answer-level hidden representation, as in CODI, when all major experimental factors are controlled?

Novel contribution

The project will go beyond final accuracy by testing whether KAVA's trajectory supervision better preserves intermediate reasoning representations, whether CODI's weaker constraint improves flexibility or OOD robustness, and how much internal supervision is actually necessary.

Concise pitch for the professor

I want to conduct a controlled comparison of CODI and KAVA as two different supervision paradigms for latent reasoning. CODI aligns an answer-level hidden representation, while KAVA aligns a compressed KV trajectory across latent steps. Rather than only testing which obtains higher accuracy, I want to study when trajectory supervision is necessary, whether it produces more recoverable intermediate reasoning, and whether endpoint supervision offers greater flexibility or out-of-distribution robustness.

Expected outcome

The strongest outcome is not necessarily a universal winner. A valuable conclusion would identify the regimes where each method is preferable and provide internal evidence explaining the difference.

Decision rule

KAVA is better only if its gains remain after matching data, architecture, latent budget, training compute, seeds, and tuning effort - and if those gains can be linked to a meaningful difference in internal reasoning rather than implementation details.

12. Core Papers

CODI: Shen et al. (2025), *Compressing Chain-of-Thought into Continuous Space via Self-Distillation*, EMNLP 2025.

KAVA: Kuzina et al. (2026), *Latent Reasoning via Compressed KV-Cache Distillation*, ICLR 2026.

Supporting context: R-KV, LongFlow, and the survey *Reasoning on a Budget* motivate the broader problem of efficient and adaptive test-time reasoning.

This proposal is designed as a controlled research study, not a product-development plan. Its main objective is to isolate and explain the scientific difference between endpoint and trajectory supervision in latent reasoning.